

Representar em LPO as seguintes frases:

1. O Miau é um gato castanho.

$$\text{Gato}(\text{Miau}) \wedge \text{Castaño}(\text{Miau})$$

2. Os gatos são animais

$$\forall x \text{ Gato}(x) \rightarrow \text{Animal}(x)$$

3. Nenhum gato é cão

$$\forall x (\text{Gato}(x) \rightarrow \neg \text{Cão}(x)) \quad \{ \neg (\exists x (\text{Gato}(x) \wedge \text{Cão}(x)))$$

4. Nem todos os gatos gostam de leite

$$\exists x (\text{Gato}(x) \wedge \neg \text{Gosta}(x, \text{Leite})) \quad \{ \neg \forall x (\text{Gato}(x) \rightarrow \text{Gosta}(x, \text{Leite}))$$

5. Se algum gato gostar do Rui, então o Miau tbm gosta do Rui

$$\exists x (\text{Gato}(x) \rightarrow \text{Gosta}(x, \text{Rui})) \rightarrow \text{Gosta}(\text{Miau}, \text{Rui})$$

STRIPS

⇒ Planning : A set of action

⇒ Action { Action' }
⇒ Pré-condição } para esta ação ser realizada é necessário atingir a
pré-condição referida; sem esta pré-condição não é possível
realizar a ação

⇒ Efeito

Exemplo: Spare Tire Problem

- Pneuzinho → chassi
 - Suplente → mala
- } retirar o pneuzinho e colocá-lo na mala;
obter o suplente e colocá-lo no chassi

⇒ Estado do objetivo :
suplente → chassi
pneuzinho → mala

Ações:

- 1) Remover suplenk
- 2) Remover vazio
- 3) colocar suplenk $\rightarrow$ chassi
- 4) colocar vazio $\rightarrow$ mala

$\Rightarrow$ 1) Action (Remove (space, trunk))

- pré-condição: $At(space, trunk)$
- efeito: $\neg At(trunk, space)$

realizando as
ações nesta sequência
iremos obter o objetivo
final

$\Rightarrow$ 2) Action (Remove (flat, axle))

- pré-condição: $At(flat, axle)$
- efeito: $\neg At(flat, axle)$

$\Rightarrow$ 3) Action (Put (flat, trunk))

- pré-condição: $\neg At(flat, axle)$
- efeito: $At(flat, trunk)$

$\Rightarrow$ 4) Action (Put (space, axle))

- pré-condição: $\neg At(space, trunk)$
- efeito: $At(space, axle)$

nome: liga-carro

pré-condição: $motor_off(carro) \wedge colocada(chave, carro)$

delete: $motor_off(carro)$

add: $motor_on(carro)$

~~$motor_off(carro)$~~

$colocada(chave, carro)$

liga-carro

$motor_on(carro)$

Pesquisa A* - Algoritmo (vídeo youtube - geeksforgeeks)

=> Explicação

Considerando um grafo com múltiplos nós. Queremos chegar ao nó final a partir do inicial, o mais depressa possível.

O que esta pesquisa faz é, em cada passo escolhe um nó de acordo com um valor "f" que é igual à soma de "g" e "h". A cada passo escolhe o nó que tem o menor "f" e processa-o.

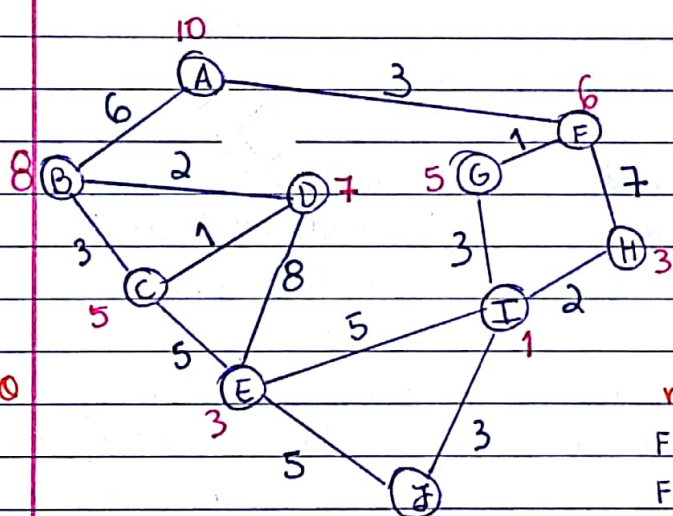
$$f(n) = g(n) + h(n)$$

$n \Rightarrow$ nó anterior do caminho

$g(n) \Rightarrow$ custo do caminho desde o nó inicial até n

$h(n) \Rightarrow$ heurística que estima o custo do caminho mais próximo até ao nó final.

=> Exemplo



n^o azuis : distância entre os nós (custo real)

n^o rosa : Heurística (custo estimado)

Objetivo : caminho com custo entre A e J

Caminho

A
↓
F
↓
G
↓
I
↓
J

nó A: 2 nós $\leq$ B
 $F(B) = 6 + 8 = 14$
 $F(F) = 3 + 6 = 9$ } $F(F) < F(B)$, é escolhido
 F como novo nó da partida

nó F: 2 nós $\leq$ G
 $F(G) = 4 + 5 = 9$ ($3 + 1 = 4$)
 $F(H) = 10 + 3 = 13$ ($3 + 7 = 10$)
 G é escolhido como novo nó

nó G: 1 nó $\rightarrow$ I
 $F(I) = 7 + 1 = 8$ ($3 + 1 + 3 + 7$) } I é o novo nó

nó I:
 $F(E) = 12 + 3 = 15$
 $F(H) = 9 + 3 = 12$
 $F(J) = 10 + 0 = 10$ } $F(J)$ é o menor
 e como J é o nó final.

FIAMOS POR AQUI!

11

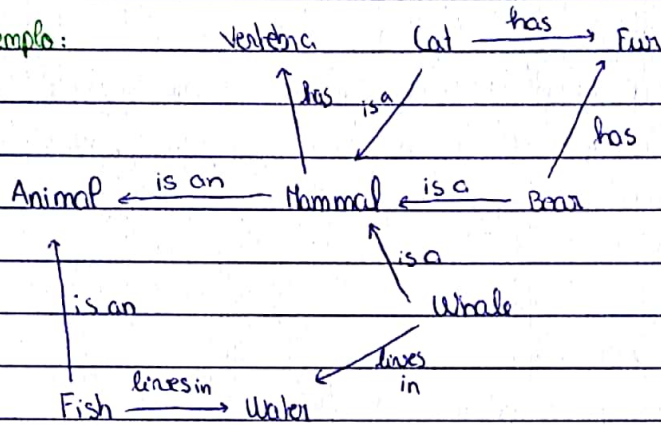
12

Redes Semânticas (Frame Network)

rede q representa relações semânticas entre conceitos

- Representação de conhecimento
- Informações proposicionais
- Gráfico direcionado e definido

Exemplo:



Herança

- ⇒ todos os membros de uma classe herdam todas as propriedades das suas superclasses
- ⇒ múltiplas heranças

Desvantagens

- ⇒ as ligações entre os objetos representam apenas relações binárias
- ⇒ não existe uma definição padrão dos nomes das ligações

Vantagens

- ⇒ capacidade de representar relações padrão para categorias
- ⇒ transmitem algum significado de forma transparente
- ⇒ redes simples e fáceis de entender
- ⇒ fáceis de traduzir em PROLOG

↓
programação lógica (IA)

Lógica de 1ª ordem

⇒ diferença em relação à lógica proposicional é a introdução da quantificação

Equivalências

$$\Rightarrow \neg \forall x P(x) (\Leftrightarrow) \exists x \neg P(x)$$

$$\Rightarrow \neg \exists x P(x) (\Leftrightarrow) \forall x \neg P(x)$$

$$\Rightarrow \forall x \forall y P(x, y) (\Leftrightarrow) \forall y \forall x P(x, y)$$

$$\Rightarrow \exists x \exists y P(x, y) (\Leftrightarrow) \exists y \exists x P(x, y)$$

$$\Rightarrow \forall x P(x) \wedge \forall x Q(x) (\Leftrightarrow) \forall x (P(x) \wedge Q(x))$$

$$\Rightarrow \exists x P(x) \vee \exists x Q(x) (\Leftrightarrow) \exists x (P(x) \vee Q(x))$$

Regras de inferência

$$\Rightarrow \exists x \forall y P(x, y) \Rightarrow \forall y \exists x P(x, y)$$

$$\Rightarrow \forall x P(x) \vee \forall x Q(x) \Rightarrow \forall x (P(x) \vee Q(x))$$

$$\Rightarrow \exists x (P(x) \wedge Q(x)) \Rightarrow \exists x P(x) \wedge \exists x Q(x)$$

Frases Atômicas

Frase atômica → Predicado (Termo, ...) | Termo = Termo

Termo → Função (Termo, ...) | constante | variável

Ex.:

- PermaEsqDe(Reizão)
- Irmãos(Reizão, RicardoCorreiaLero)
- Pai(Reizão) = Henrique

Frases complexas

Frase Complexa → Frase Atômica | (Frase Complexa conectiva Frase Complexa) | Quantificação, ..., Frase Complexa

$\neg$ Frase Complexa $\wedge \vee \Rightarrow \Leftarrow$ $\forall, \exists$

Ex.:

- Irmãos(Reizão, RicardoCorreiaLero) $\Rightarrow$ Irmãos(RicardoCorreiaLero, Reizão)
- $\neg$ Irmãos(PermaEsqDe(RicardoCorreiaLero), Reizão)
- $\forall x, y$ Irmãos(x, y) $\Rightarrow$ Irmãos(y, x)

Quantificação Universal

$\forall$ <variáveis> < frase >

Ex.: Todos os reis são pessoas

$$\forall x \text{ Rei}(x) \Rightarrow \text{Pessoa}(x)$$

Quantificação existencial

$\exists$ <variáveis> < frase >

Ex.: O Rei João tem uma coroa na cabeça

$$\exists x \text{ Elmen}(x) \wedge \text{NaCabeça}(x, \text{ReiJoão})$$

Muitos exemplos / exercícios de LPO :

1) Todos os A's são B's

$$\forall x A(x) \Rightarrow B(x)$$

2) Nenhum A é B

$$\neg \exists x A(x) \wedge B(x)$$

3) Alguns A's são B's

$$\exists x A(x) \wedge B(x)$$

4) Alguns A's não são B's

$$\exists x A(x) \wedge \neg B(x)$$

5) Somente os A's são B's

$$\forall x B(x) \Rightarrow A(x)$$

6) Nem todos os A's são B's

$$\exists x A(x) \wedge \neg B(x)$$

7) Todos os A's não são B's

$$\neg \exists x A(x) \wedge B(x)$$

8) Todas as pessoas gostam de outra pessoa

$$\forall x \text{ Pessoa}(x) \Rightarrow \exists y \text{ Pessoa}(y) \wedge \text{gosta}(x, y) \wedge \neg(x=y)$$

9) Existe uma pessoa de quem todas as outras pessoas gostam

$$\exists x \text{ Pessoa}(x) \wedge \forall y \text{ Pessoa}(y) \wedge \neg(x=y) \Rightarrow \text{gosta}(y, x)$$

10) O João frequenta a cadeira de IA ou PE

$$\text{Frequenta}(\text{João}, \text{IA}) \vee \text{Frequenta}(\text{João}, \text{PE})$$

11) O Rui frequenta ou IA ou PE.

$$\text{Frequenta}(\text{Rui}, \text{IA}) \Leftrightarrow \neg \text{Frequenta}(\text{Rui}, \text{PE})$$

Lógica Proposicional

Sintaxe: símbolos proposicionais $S, S_1, S_2, \dots$ representam fatos e são frases da linguagem

- Se $\underline{S}$ é uma frase, $\neg S$ é uma frase (negação)
- Se $\underline{S_1}$ e $\underline{S_2}$ são frases, $S_1 \wedge S_2$ é uma frase (conjunção)
- Se $\underline{S_1}$ e $\underline{S_2}$ são frases, $S_1 \vee S_2$ é uma frase (disjunção)
- Se $\underline{S_1}$ e $\underline{S_2}$ são frases, $S_1 \Rightarrow S_2$ é uma frase (implicação)
- Se $\underline{S_1}$ e $\underline{S_2}$ são frases, $S_1 \Leftrightarrow S_2$ é uma frase (equivalência)

Vantagens

- Lógica proposicional é declarativa
- permite informação parcial / disjuntiva / negada
- composta
- é independente do contexto

Desvantagens

- poder de expressividade limitado

Forma normal clausal

Ex.: $\forall x (\exists z (\neg P(x, z) \vee \exists y P(y, x))) \rightarrow$ forma normal prenex

$\rightarrow \forall x (\neg P(x, f(x)) \vee P(g(x), x)) \rightarrow$ Skolemizar: substituir z por $f(x)$ e y por $g(x)$

$\neg P(x, f(x)) \vee P(g(x), x) \rightarrow$ Remover quantificadores

Forma normal conjuntiva

conj. de cláusulas: $\{ \neg P(x, f(x)) \vee P(g(x), x) \}$

Ex. 2: $\forall x (\neg \forall y (P(x, y) \wedge \forall z \neg R(y, z)) \wedge \forall y \neg P(x, y))$

Passo 1) Colocar a fórmula na forma normal da negação

$$\forall x (\exists y \neg (P(x, y) \wedge \forall z \neg R(y, z)) \wedge \forall y \neg P(x, y))$$

$$\forall x (\exists y (\neg P(x, y) \vee \neg \forall z \neg R(y, z)) \wedge \forall y \neg P(x, y))$$

$$\forall x (\exists y (\neg P(x, y) \vee \exists z R(y, z)) \wedge \forall y \neg P(x, y))$$

Passo 2) Skolemizar

$$\forall x \exists y_1 \exists z \forall y_2 ((\neg P(x, y_1) \vee R(y_1, z)) \wedge \neg P(x, y_2))$$

Substituindo y_1 por $f(x)$ e z por $g(x)$:

$$\forall x \forall y_2 ((\neg P(x, f(x)) \vee R(f(x), g(x))) \wedge \neg P(x, y_2))$$

Passo 3) Remover quantificadores

$$((\neg P(x, f(x)) \vee R(f(x), g(x))) \wedge \neg P(x, y_2))$$

conj. de cláusulas: $\{ \neg P(x, f(x)) \vee R(f(x), g(x)), \neg P(x, y_2) \}$

Forma normal conjuntiva

Ex.:

$$\cdot A \wedge B$$

$$\cdot \neg A \wedge (B \vee C)$$

$$\cdot (A \vee B) \wedge (\neg B \vee C \vee \neg D) \wedge (D \vee \neg E)$$

$$\cdot (\neg B \vee C)$$

Fórmulas $\implies$ CNF

$$\neg(B \vee C)$$

$$\neg B \wedge \neg C$$

$$(A \wedge B) \vee C$$

$$(A \vee C) \wedge (B \vee C)$$

$$A \wedge (B \vee (D \wedge E))$$

$$A \wedge (B \vee D) \wedge (B \vee E)$$

KIF - Knowledge Interchange Format

=> projetado para permitir que os sistemas compartilhem e reutilizem informações de sistemas baseados no conhecimento

=> intercâmbio de conhecimento entre sistemas

=> semântica declarativa

Redes Bayes → conhecimento probabilístico
→ grafo acíclico dirigido

Teorema de Bayes

=> a relação entre uma probabilidade condicionada e a sua inversa

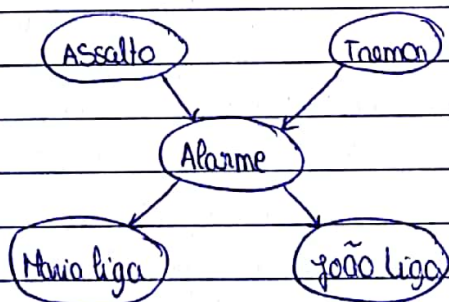
$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}$$

Ex. prático:

- Sistema de alarme ^{→ A} instalado em casa
 - ativado qd há assalto; ^{→ AS}
 - responde qd há pequenos tremores de terra; ^{→ T}
- 2 vizinhos q telefonam qd ouviram o alarme

ML ← Maria: costuma ouvir música alta e por vezes ã ouve o alarme

JL ← João: telefona sempre mas por vezes confunde o alarme c/ o toque do telemóvel



Exemplo dos slides

Pesquisa em árvore $\rightarrow$ estrutura de dados em árvore utilizada para localizar chaves específicas dentro de um conjunto.

Para q uma árvore funcione como uma pesquisa, a chave para cada nó deve ser maior do q qqr chave nas sub-árvores à esquerda e menor do q qqr chave nas sub-árvores à direita.

Vantagem: tempo de pesquisa eficiente

Pesquisa IDA* - interactive deeping

$\downarrow$ $\rightarrow$ pesquisa em profundidade $\leftarrow$
versão restrita de memória da pesquisa A* (usa menos memória)

Vantagens

- encontra sempre a solução ideal desde q exista uma heurística admissível
- heurística ñ é necessária, é usada para acelerar o processo
- várias heurísticas podem ser adicionadas ao algoritmo sem alterar código.
- o custo de cada movimento pode ser ajustado nos algoritmos
- usa mt menos memória

Desvantagens

- ñ acompanha os nós visitados; explora nós repetidos
- $\oplus$ lento
- requer $\oplus$ potência e tempo de processamento

Para cada iteração:

- realizar 1a pesquisa em profundidade, comparando o valor $f(n)$ do nó com um valor de corte
- parar de procurar o ramo atual qd o corte é excedido
- o ponto de corte começa como o custo estimado do nó inicial
- no final da iteração, aumentar o corte, configurando-o para o menor valor $f(n)$ q excedeu o corte durante esta iteração

$\Rightarrow$ completo

$\Rightarrow$ ótimo em termos de tempo, espaço e custo da solução

$\Rightarrow \oplus$ simples de implementar (que o A*)

álculo do custo do caminho

custo uniforme: custo percorrido

gulosa: custo restante estimado

A^* : custo percorrido + custo restante estimado

Pesquisa gulosa:

- completa desde que não se visite nós repetidos
- não é ótima
- ⊕ rápida que o custo uniforme (expande menos nós)

Pesquisa A^* :

- completa → caso a heurística seja admissível
- ótima

EXERCÍCIOS

Lógica de 1ª ordem

Nem todos os estudantes se inscrevem simultaneamente a IA e SO.

$$\exists x \text{ Estudante}(x) \wedge \neg \text{Inscrever}(IA, SO)$$

Só um aluno chumbou a história e biologia

$$\exists x \text{ Aluno}(x) \wedge \text{Aluno}(História, Biologia) \wedge \text{Chumbou}(História, Biologia)$$

Existe 1 sportinguista q gosta de todos os beifiquistas que não são espertos

$$\exists x S(x) \wedge \forall y B(y) \wedge \neg E(y) \Rightarrow G(x, y)$$

Qualquer animal tem 1 progenitor

$$\forall x A(x) \Rightarrow \exists y A(y) \wedge P(y, x)$$

Se (P) é o progenitor de (A) e (A) pertence a uma espécie (E) então (P) hom pertence a (E).

$$\forall x \forall y \forall z \exists x A(x) \wedge P(x, y) \wedge E(x, z) \Rightarrow E(y, z)$$

x - animal

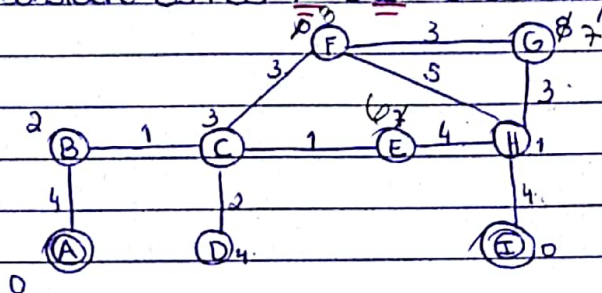
y - progenitor

z - espécie

Não existem mamutes vivos em 1518.

$$\neg (\exists x A(x) \wedge \text{Mute}(x, Mamute) \wedge \text{Vivo}(x, 1518))$$

Considere os nós A e I como soluções possíveis



a) A Heurística apresentada é admissível? Em caso negativo, faça as alterações necessárias para fazer a que passe a sê-lo.

Para a heurística ser admissível o custo $\geq$ custo estimado

	A-B	A-C	A-D	A-E	A-F	A-G	A-H
custo	4	5	7	6	8	11	10
custo estimado	2	3	4	7	10	8	1

↑ ↑

custo estimado > custo !!!

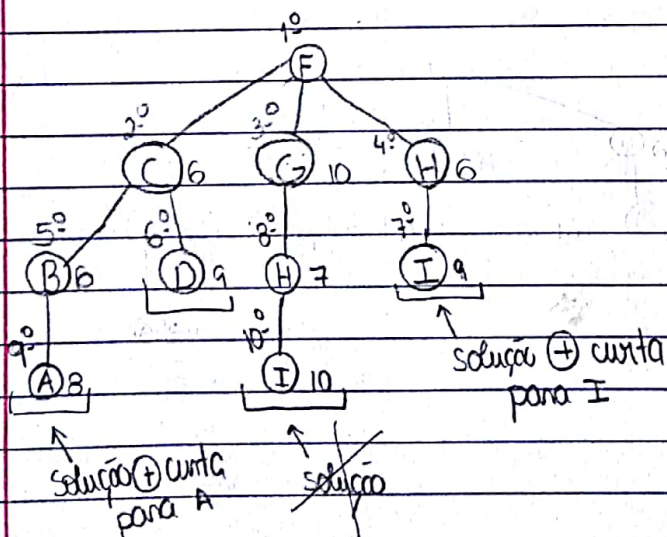
	I-B	I-C	I-D	I-E	I-F	I-G	I-H
custo	10	9	11	8	9	7	4
custo estimado	2	3	4	7	10	8	1

↑ ↑

custo estimado > custo !!!

a heurística não é admissível para os nós : E, F, G

b) Desenhe a árvore de pesquisa gerada pela estratégia A^* , tomando como estado inicial o estado F. (Sem repetição de estados)



$$RH = \frac{N-1}{x} = \frac{10-1}{5} = \frac{9}{5}$$

$$N = \frac{B^{d+1}-1}{B-1} (=) 10 = \frac{B^5-1}{B-1}$$

$$(\Rightarrow) 10B - 10 = B^5 - 1$$

$$(\Rightarrow) 10B - B^5 = 9$$

$$(\Rightarrow) B - B^5 = \frac{9}{10}$$

c) Calcule o fator de ramificação média da árvore gerada

$$RM = \frac{N-1}{x} = \frac{11-1}{4} = \frac{10}{4} = 2,5$$

$N = n^{\circ}$ total nós da árvore

$x = n^{\circ}$ de nós expandidos

$d = \text{profundidade}$

d) Calcule o fator de ramificação efetivo da árvore gerada.

$$N = \frac{B^{d+1} - 1}{B - 1} (=) \frac{B^4 - 1}{4 - 1} = 11 (=) B^4 - 1 = 33 (=) B^4 = 34 (=) B \approx 2,3$$

Teste Modelo IA

I

① Analise as seguintes funções em Python e explique o que fazem

a) def f(x):

if x == []:

return 0

if x[0] > 0:

return x[0] + f(x[1:])

return f(x[1:])

=> A função recebe uma lista como argumento;

Se a lista se encontrar vazia, a função retorna 0;

Se o 1º valor for maior que 0 é retornada a soma dos elementos da lista, recursivamente.

b) def g(x):

if x == []:

return []

y = g(x[1:])

return y + [x[0] + z for z in y]

=> Dada uma lista, se a lista

estiver vazia, é retornada uma lista com uma lista vazia

A variável y recebe os valores da lista (recursivamente)

② Programar uma função que, dada uma lista de variáveis, gere todas as combinações de valores possíveis.

def f(l):

if l == []:

return []

x = f(l[1:])

nr = x[0]

return [(nr, true) + i for i in x] + [(nr, false) + i for i in x]

II - Escolhas Múltiplas

1.

a) "Todos os livros de BD têm capa dura" - LPO

$$\forall x (\text{Livro}(x) \wedge \text{BD}(x)) \Rightarrow \neg \text{CapaDura}(x)$$

$$\forall x \text{BD}(x) \Rightarrow \neg \text{Capa}(x, \text{Dura})$$

$$\forall x \text{Livro}(x) \vee \text{BD}(x) \Rightarrow \neg \text{Capa}(x, \text{Dura})$$

$$\forall x \text{Livro}(x) \wedge \text{BD}(x) \Rightarrow \neg \text{Capa}(x, \text{Dura})$$

Nenhuma das anteriores

X

$$R.: \forall x \text{Livro}(x) \wedge \text{BD}(x) \Rightarrow \text{CapaDura}(x)$$

b) "A melhor nota a Português foi a da Ana" - LPO

$$\forall x \text{Nota}(\text{Ana}, \text{PT}) > \text{Nota}(x, \text{Português}) \wedge \text{Aluno}(x)$$

$$\forall x, y, z \text{Nota}(\text{Ana}, \text{PT}, y) > \text{Nota}(x, \text{PT}, z) \wedge y > z$$

$$\forall x \text{Nota}(\text{Ana}, \text{PT}) \geq \text{Nota}(x, \text{Português}) \vee \text{Aluno}(x)$$

$$\forall x \text{Aluno}(x) \Rightarrow (\text{Nota}(\text{Ana}, \text{PT}) \geq \text{Nota}(x, \text{PT}))$$

Nenhuma das anteriores

X

c) Pesquisa por melhorias sucessivas é:

Uma técnica de pesquisa para resolução de problemas de atribuição

" " para combinação de heurísticas

" " de pesquisa para otimização de soluções

Passo particular do crescimento simulado em q a evolução da temperatura faz lembrar uma paisagem de montanhas

Nenhuma das anteriores

X

d) Uma consequência lógica do conj. $\{A \vee B, \neg B \vee C \vee D, \neg A, \neg D\}$ é

$B \wedge A$

C

A

$A \vee D$

Nenhuma das anteriores

X

$A \vee B$

~~$\neg B \vee C \vee D$~~

~~$A \vee C \vee D$~~

~~$\neg A$~~

~~$C \vee D$~~

~~$\neg D$~~

C //

e) Os operadores STRIPS são:

Formato de representação de ações para sistemas reativos / estado interno
 " " " de transições de estados para pesquisa de melhorias sucessivas
 Mecanismos de modificação da solução em pesquisa por reconhecimento simbólico
 " para geração de planos no mundo dos blocos
 Nenhuma das anteriores

X	

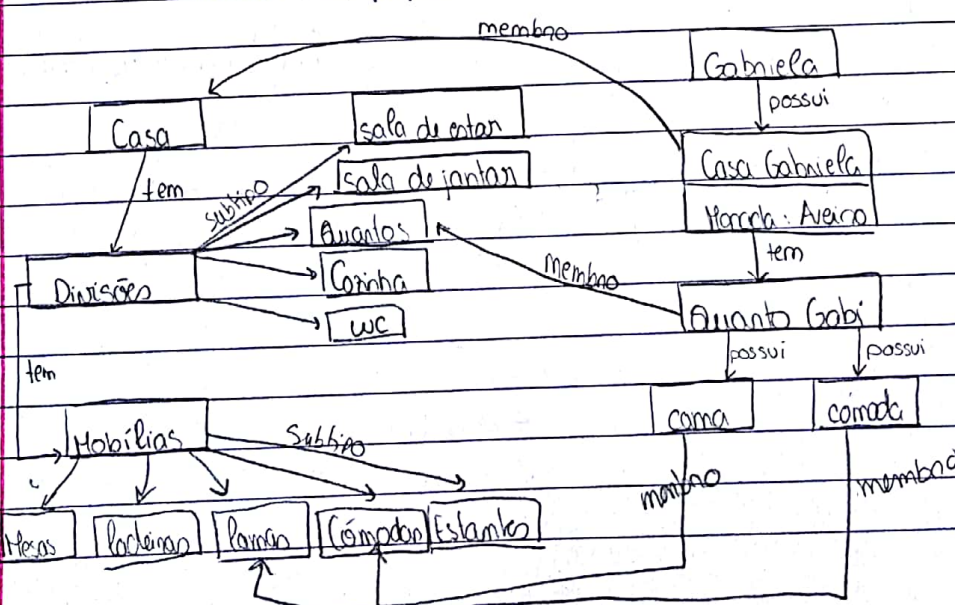
2. Semelhanças e diferenças entre a pesquisa em árvore em profundidade e a pesquisa por montanhismo.

Pesquisa em árvore em profundidade: consiste em avaliar 1º um dos nós e, se este não for solução, avaliar um dos seus filhos. Repete-se este passo até chegar a um nó que não possui filhos ou cujos filhos já foram avaliados, sendo então designado o melhor ao nó pai.

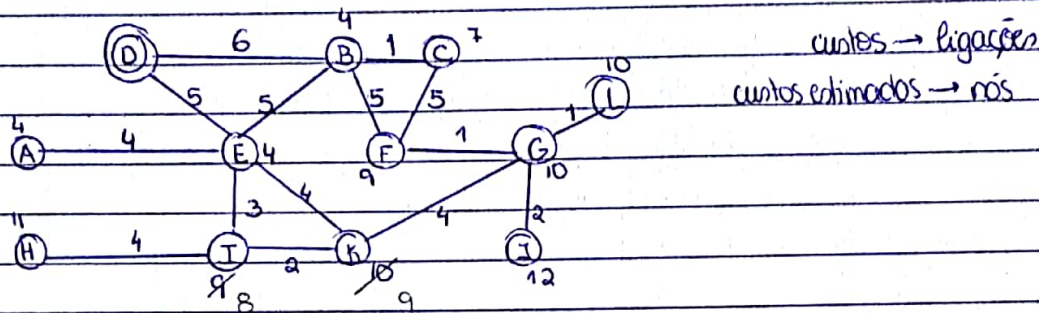
Montanhismo: é semelhante, diferenciando-se em: escolher sempre o sucessor com melhor valor da função de avaliação, não há retrocessos e qd o valor da função no nó atual é superior ao valor da função em qqn dos seus sucessores, a pesquisa pára.

3. Representar numa rede semântica

Casas → Divisões: sala de estar; sala de jantar; quartos de dormir, cozinhas e quartos de banho
 Gabriela → Casa em Aveiro
 dono 1 quarto c/ 12 cama
 1 cozinha
 peças mobiliário: mesas, cadeiras, camas, cómodas, estantes



4. D - estado objetivo



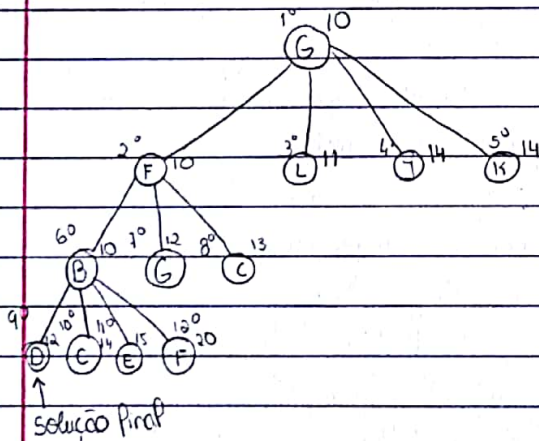
a) Para a heurística ser admissível o custo tem que ser menor ou igual ao custo estimado

$$\text{custo} \geq \text{custo estimado}$$

	D-A	D-B	D-C	D-E	D-F	D-G	D-H	D-I	D-K	D-L
custo	9	6	7	5	11	12	12	14	9	8
custo estimado	4	4	7	4	9	10	11	12	10	9

~~custo < custo estimado~~

b) G é o estado inicial ; apresenta a tree search $\Rightarrow$ pesquisa A* c/ repetição de estados



nó G : K ; F ; J ; L

$$F(K) = 4 + 10 = 14$$

$$F(F) = 1 + 9 = 10$$

$$F(J) = 2 + 12 = 14$$

$$F(L) = 1 + 10 = 11$$

nó F : B ; C ; G

$$F(B) = 6 + 4 = 10 \quad (5+1=6)$$

$$F(C) = 6 + 7 = 13 \quad (5+1=6)$$

$$F(G) = 2 + 10 = 12 \quad (1+1=2)$$

nó B : D ; E ; F ; C

$$F(D) = 12 + 0 = 12$$

$$F(E) = 11 + 4 = 15$$

$$F(F) = 11 + 9 = 20$$

$$F(C) = 7 + 7 = 14$$

5. Considere um veículo autônomo q se movimenta num ambiente estruturado em nós e ligações, ou seja, estruturado como um grafo. As ligações correspondem a ruas. Os nós representam confluências de 1/4 ruas. Em cada nó pode haver 0/+ parques de estacionamento. As ruas começam e terminam em nós adjacentes no grafo. O agente é capaz de realizar as seguintes ações:

- atravessar
- estacionar
- percorrer
- sair

a) conj. de predicados LPO - valores possíveis

Rua(A,B) $\rightarrow$ rua que liga A a B

Parque(p,A) $\rightarrow$ parque em A

Estacionado(p) $\rightarrow$ veículo estacionado no parque

NaRua(A,B) $\rightarrow$ andar na rua q vai de A a B

b) conj. de operações STRIPS

Atravessar(A,B,C):

pré-condição: Rua(A,B) $\wedge$ Rua(A,C) $\wedge$ NaRua(A,B)

positivo: NaRua(A,C)

negativo: NaRua(A,B)

Estacionar(A,B,p):

pré-condição: Rua(A,B) $\wedge$ NaRua(A,B) $\wedge$ Parque(p,A)

positivo: Estacionado(p)

negativo: NaRua(A,B)

Percorrer(A,B):

pré-condição: Rua(A,B) $\wedge$ NaRua(A,B)

positivo: NaRua(B,A)

negativo: NaRua(A,B)

Sair(p,A,B):

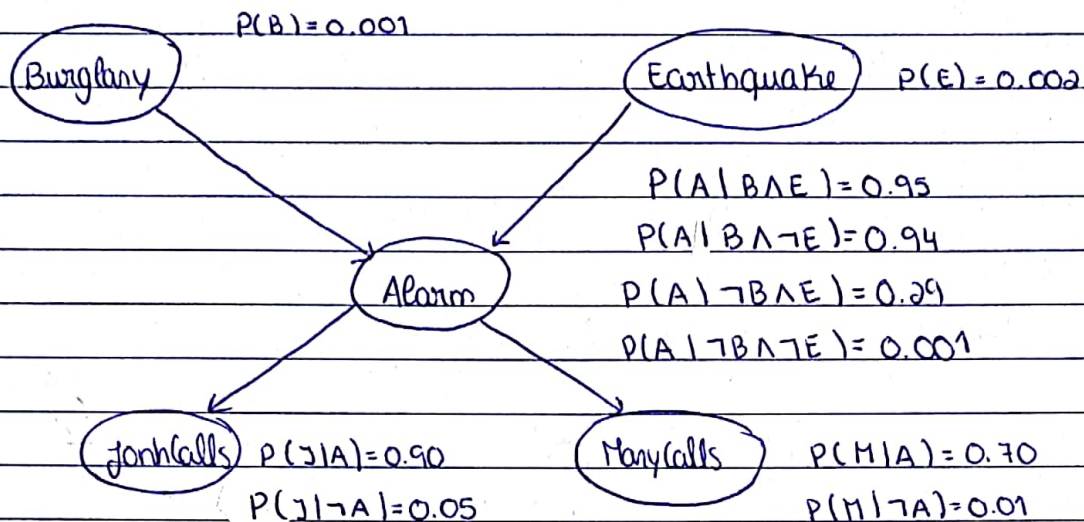
pré-condição: Estacionado(p) $\wedge$ Rua(A,B)

positivo: NaRua(A,B)

negativo: Estacionado(p)

Redes de bayes - exemplo

→ Modelo gráfico probabilístico



→ Probabilidade de o alarme tocar e o João e a Maria ambos ligarem quando $\neg$ há nenhum terremoto

$$\begin{aligned} & P(A \wedge J \wedge M \wedge \neg B \wedge \neg E) \\ &= P(J|A) \times P(M|A) \times P(A|\neg B \wedge \neg E) \times P(\neg B) \times P(\neg E) \\ &= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998 \\ &= 0.000628 \end{aligned}$$